

02393 Programming in C++

Module 2:

Basic C++ and Data Types

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Compiling C++ programs
- Pointers
- Arrays
- More about data types
- Enumerations
- Structures
- Miscellaneous features
- Lab

A recap from the first lecture

- The structure of a C++ program
 - #include directives, the main function, user-defined functions
- Simple input/output
 - cin, cout
- Variables, values and types
 - string, int, double, float, bool
- Expressions
 - some numeric and boolean operators and math functions
- Statements
 - if, while, for

Compiling C++ programs

1. Edit the .cpp source code file (e.g., program.cpp) and save it
 - Microsoft Visual Studio Code is the recommended editor
2. Compile program.cpp producing the executable file program.exe
 - With GCC, the run the command: `g++ program.cpp -o program.exe`
 - With MacOS and older versions of Xcode tools, run the command:
`g++ program.cpp -o program.exe -std=c++17`
3. If you don't get compilation errors, run the resulting executable:
 - `./program.exe`

Checking the compiler output

```
$ g++ test.cpp -o test.exe
test.cpp: In function 'int main()':
test.cpp:8:29: error: 'miss' was not declared in this scope
   8 |         cout << "Hello " << miss << endl;
     |                               ^~~~~
```

- Variable “miss” was not declared

```
$ g++ test.cpp -o test.exe
test.cpp: In function 'int main()':
test.cpp:8:33: error: expected ';' before 'return'
   8 |         cout << "Hello " << endl
     |                                   ^
   9 |         return 0;
     |         ~~~~~
```

- A semicolon is missing

```
$ g++ test.cpp -o test.exe
test.cpp: In function 'int main()':
test.cpp:7:9: error: 'cout' was not declared in this scope
   7 |         cout << "Hello " << endl
     |         ^~~~~
test.cpp:3:1: note: 'std::cout' is defined in header '<iostream>'; did you forget to '#include <iostream>'?
   2 | #include <string>
   +++ |+#include <iostream>
   3 |
test.cpp:7:29: error: 'endl' was not declared in this scope
   7 |         cout << "Hello " << endl
     |                               ^~~~~
test.cpp:3:1: note: 'std::endl' is defined in header '<iostream>'; did you forget to '#include <iostream>'?
   2 | #include <string>
   +++ |+#include <iostream>
   3 |
```

- iostream library was not included

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << " The value of p is " << p <<
5     " and points to value " << *p << endl;
6 p = &x;
7 cout << " The value of p is " << p <<
8     " and points to value " << *p << endl;
```

- “Normal” variables contain a value

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1004

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << " The value of p is " << p <<
5     " and points to value " << *p << endl;
6 p = &x;
7 cout << " The value of p is " << p <<
8     " and points to value " << *p << endl;
```

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1004

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << " The value of p is " << p <<
5     " and points to value " << *p << endl;
6 p = &x;
7 cout << " The value of p is " << p <<
8     " and points to value " << *p << endl;
```

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address
- When accessing a pointer by its name, we access the memory address

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1004

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << " The value of p is " << p <<
5     " and points to value " << *p << endl;
6 p = &x;
7 cout << " The value of p is " << p <<
8     " and points to value " << *p << endl;
```

Address	Contents
0x1000	x = 5
0x1004	y → 42
0x1008	p → 0x1004

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address
- When accessing a pointer by its name, we access the memory address
- To access the contents of the memory address, one needs to prepend * to the pointer variable name

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << "The value of p is " << p <<
5     " and points to value " << *p << endl;
6 p = &x;
7 cout << "The value of p is " << p <<
8     " and points to value " << *p << endl;
```

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1004

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address
- When accessing a pointer by its name, we access the memory address
- To access the contents of the memory address, one needs to prepend * to the pointer variable name
- To get the memory address where the contents of a variable reside, one needs to prepend & to the variable name

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << " The value of p is " << p <<
5   " and points to value " << *p << endl;
6 p = &x;
7 cout << " The value of p is " << p <<
8   " and points to value " << *p << endl;
```

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1000

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address
- When accessing a pointer by its name, we access the memory address
- To access the contents of the memory address, one needs to prepend * to the pointer variable name
- To get the memory address where the contents of a variable reside, one needs to prepend & to the variable name
 - It can be used to assign that address to a pointer

Arrays in C++

```
1 int a[] = {1, 2, 3};  
2 double b[3];  
3  
4 cout << a[0] + 1 << endl;  
5 b[2] = 3.14;
```

- An array is a collection of values of a same type
- We can easily define arrays when their size is known at compile-time
 - We can initialize arrays when they are created

Arrays in C++

```
1 int a[] = {1, 2, 3};  
2 double b[3];  
3  
4 cout << a[0] + 1 << endl;  
5 b[2] = 3.14;
```

- An array is a collection of values of a same type
- We can easily define arrays when their size is known at compile-time
 - We can initialize arrays when they are created
 - We can defer the initialization later:
 - The array is created, but its contents are uninitialized

Arrays in C++

- To access array elements, we use indexing, starting with 0
 - Reading contents from the first element of array a

```
1  int a[] = {1, 2, 3};  
2  double b[3];  
3  
4  cout << a[0] + 1 << endl;  
5  b[2] = 3.14;
```

Arrays in C++

```
1 int a[] = {1, 2, 3};  
2 double b[3];  
3  
4 cout << a[0] + 1 << endl;  
5 b[2] = 3.14;
```

- To access array elements, we use indexing, starting with 0
 - Reading contents from the first element of array a
 - Writing contents to the third element of array b

Arrays in C++

```
1 int a[] = {1, 2, 3};  
2 a[3000] = 6;  
3 cout << "Hello " << a[3000] << endl;
```

- Arrays do not contain information on their size
 - You need to store it elsewhere
- You must pay attention not to index an array beyond its last element
 - If you don't, you will end up reading or writing memory areas which belong to other variables, or to other programs!
 - Therefore, the execution may lead to unspecified results or crashes!

Arrays in C++

```
1 int a[] = {100 , 101 , 102};
2 int *p;
3 p = a;
4 cout << " The value of a is " << a <<
5     " and points to value " << a[0] << endl;
6 cout << " The value of p is " << p <<
7     " and points to value " << *p << endl;
8 cout << "p[0]= " << p[0] << " p[1]= " << p[1] <<
9     " p[2]= " << p[2] << endl;
```

- A variable of type array is just a pointer to the beginning of the array

Address	Contents
0x1000	100
0x1004	101
0x1008	102
0x1012	a = 0x1000
0x1016	p = 0x1000

Arrays in C++

```
1 int a[] = {100 , 101 , 102};
2 int *p;
3 p = a;
4 cout << " The value of a is " << a << endl;
5     " and points to value " << a[0] << endl;
6 cout << " The value of p is " << p << endl;
7     " and points to value " << *p << endl;
8 cout << "p[0]= " << p[0] << " p[1]= " << p[1] << endl;
9     " p[2]= " << p[2] << endl;
```

- A variable of type array is just a pointer to the beginning of the array

Address	Contents
0x1000	100
0x1004	101
0x1008	102
0x1012	a = 0x1000
0x1016	p = 0x1000

Arrays in C++

```
1 int a[] = {100 , 101 , 102};
2 int *p;
3 p = a;
4 cout << " The value of a is " << a <<
5   " and points to value " << a[0] << endl;
6 cout << " The value of p is " << p <<
7   " and points to value " << *p << endl;
8 cout << "p[0]= " << p[0] << " p[1]= " << p[1] <<
9   " p[2]= " << p[2] << endl;
```

- A variable of type array is just a pointer to the beginning of the array

Address	Contents
0x1000	100
0x1004	101
0x1008	102
0x1012	a = 0x1000
0x1016	p = 0x1000

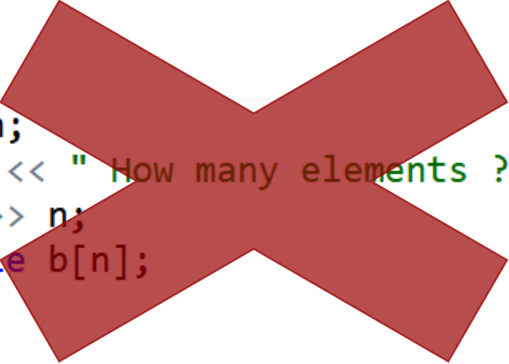
Arrays in C++

```
1 int a[] = {100, 101, 102};
2 int *p;
3 p = a;
4 cout << " The value of a is " << a <<
5     " and points to value " << a[0] << endl;
6 cout << " The value of p is " << p <<
7     " and points to value " << *p << endl;
8 cout << "p[0]= " << p[0] << " p[1]= " << p[1] <<
9     " p[2]= " << p[2] << endl;
```

- A variable of type array is just a pointer to the beginning of the array

Address	Contents
0x1000	100
0x1004	101
0x1008	102
0x1012	a = 0x1000
0x1016	p = 0x1000

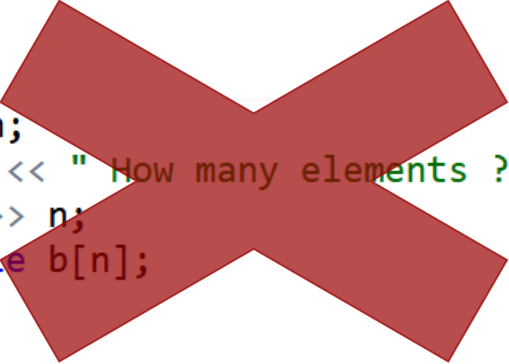
Arrays in C++



```
1 int n;  
2 cout << "How many elements?" << endl;  
3 cin >> n;  
4 double b[n];
```

- What to do if the size of the array is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!

Arrays in C++

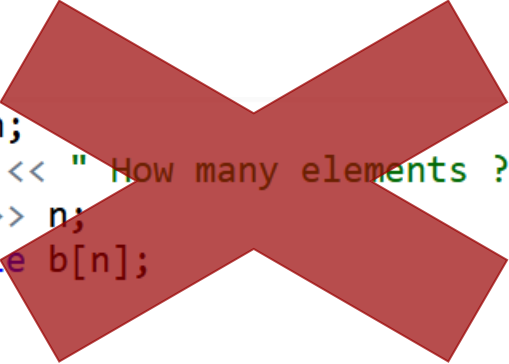


```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double b[n];
```

```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double *b = new double[n];  
5 // Use b as a normal array...  
6 delete[] b;  
-
```

- What to do if the size of the array is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array

Arrays in C++

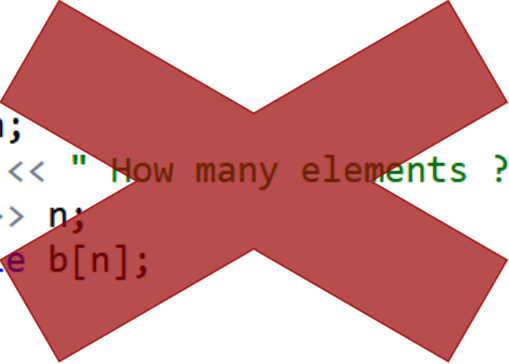


```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double b[n];
```

```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double *b = new double[n];  
5 // Use b as a normal array...  
6 delete[] b;  
-
```

- What to do if the size of the array is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array
 - new allocates an array of size n

Arrays in C++



```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double b[n];
```

```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double *b = new double[n];  
5 // Use b as a normal array...  
6 delete[] b;
```

- What to do if the size of the array is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array
 - new allocates an array of size n
 - delete deallocates the memory area occupied by the array

Break

We resume at 19:30

Data types precision and conversions

- long double
- double
- float
- unsigned long
- long
- unsigned int
- int
- unsigned short
- short
- char

Most precise



Least precise

- C++ numeric data types differ in memory usage and precision
 - They all have minimum and maximum values
 - The compiler may perform automatic conversions toward the “largest” type
 - We can explicitly cast (convert) values across types (but be careful!)
 - floats and doubles are sometimes approximated, leading to rounding errors

Enumerations

```
1  enum Direction {NORTH, EAST, SOUTH, WEST};
2  Direction dir = NORTH;
3
4  void tellDirection(Direction dir) {
5  switch (dir) {
6      case NORTH:
7          cout << " North " << endl;
8          break;
9      case WEST:
10         cout << " West " << endl;
11         break;
12     case SOUTH:
13         cout << " South " << endl;
14         break;
15     case EAST:
16         cout << " East " << endl;
17         break;
18     default:
19         cout << " WARNING : invalid direction : " << dir << endl;
20     }
21 }
```

- An enumeration is a user-defined data type with values limited to a given finite set

Enumerations

```
1 enum Direction {NORTH, EAST, SOUTH, WEST};
2 Direction dir = NORTH;
3
4 void tellDirection(Direction dir) {
5     switch (dir) {
6         case NORTH:
7             cout << " North " << endl;
8             break;
9         case WEST:
10            cout << " West " << endl;
11            break;
12        case SOUTH:
13            cout << " South " << endl;
14            break;
15        case EAST:
16            cout << " East " << endl;
17            break;
18        default:
19            cout << " WARNING : invalid direction : " << dir << endl;
20    }
21 }
```

- An enumeration is a user-defined data type with values limited to a given finite set
- Very handy for representing symbolic information and using the switch statement

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created
- We can defer the initialization later:
 - The structure is created, but its contents are uninitialized

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2:  
9     e2.name = "Margaret";  
10    e2.surname = "Hamilton";  
11    e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created
- We can defer the initialization later:
 - The structure is created, but its contents are uninitialized
- To access a member of a structure e1, we use the syntax e1.member

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created
- We can defer the initialization later:
 - The structure is created, but its contents are uninitialized
- To access a member of a structure e1, we use the syntax e1.member
- If we have a pointer p to a structure, to access a member we can use:
 - (*p).member

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created
- We can defer the initialization later:
 - The structure is created, but its contents are uninitialized
- To access a member of a structure e1, we use the syntax e1.member
- If we have a pointer p to a structure, to access a member we can use:
 - (*p).member
 - p->member

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6  
7 employee employees[2];  
8  
9 employees[0] = {" John ", "Doe", 50000};  
10  
11 employees[1].name = "Margaret";  
12 employees[1].surname = "Hamilton";  
13 employees[1].salary = 500000;
```

- Being a (user-defined) data type, we can declare arrays of structures
- Handy for representing multiple records (e.g., an employee registry, an inventory, etc.).

Miscellaneous features

- The preprocessor directive `#define` assigns 10 to the symbol `MAXIMUM`
 - `MAXIMUM` is NOT a variable

```
1 #define MAXIMUM 10
2
3 int main () {
4     for (int i = 0; i < MAXIMUM; i++) {
5         string oddOrEven = ((i % 2) == 0) ? "even" : "odd";
6         cout << "The number " << i << " is " << oddOrEven << endl;
7     }
8     return 0;
9 }
```

Miscellaneous features

- The preprocessor directive `#define` assigns 10 to the symbol `MAXIMUM`
 - `MAXIMUM` is NOT a variable
 - Whenever `MAXIMUM` is used in the code, the compiler replaces it with 10
 - It works like find&replace
 - Handy to make the code parametric

```
1  #define MAXIMUM 10
2
3  int main () {
4      for (int i = 0; i < MAXIMUM; i++) {
5          string oddOrEven = ((i % 2) == 0) ? "even" : "odd";
6          cout << "The number " << i << " is " << oddOrEven << endl;
7      }
8      return 0;
9  }
```

Miscellaneous features

- The preprocessor directive `#define` assigns 10 to the symbol `MAXIMUM`
 - `MAXIMUM` is NOT a variable
 - Whenever `MAXIMUM` is used in the code, the compiler replaces it with 10
 - It works like find&replace
 - Handy to make the code parametric
- The `++` operator increments a numeric variable of 1

```
1  #define MAXIMUM 10
2
3  int main () {
4      for (int i = 0; i < MAXIMUM; i++) {
5          string oddOrEven = ((i % 2) == 0) ? "even" : "odd";
6          cout << "The number " << i << " is " << oddOrEven << endl;
7      }
8      return 0;
9  }
```

Miscellaneous features

```
1 #define MAXIMUM 10
2
3 int main () {
4     for (int i = 0; i < MAXIMUM; i++) {
5         string oddOrEven = ((i % 2) == 0) ? "even" : "odd";
6         cout << "The number " << i << " is " << oddOrEven << endl;
7     }
8     return 0;
9 }
```

- The preprocessor directive `#define` assigns 10 to the symbol `MAXIMUM`
 - `MAXIMUM` is NOT a variable
 - Whenever `MAXIMUM` is used in the code, the compiler replaces it with 10
 - It works like find&replace
 - Handy to make the code parametric
- The `++` operator increments a numeric variable of 1
- Conditional expressions are a compact way to express binary decisions:
 - If `i` modulo 2 is zero, “even” is assigned to `oddOrEven`
 - If not, “odd” is assigned to `oddOrEven`

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 2
 - Ask questions if something is unclear (including on Assignment 1)

DTU

